

Diego Murillo | Digital Business Intelligence



Cracking the Code (No Coding Required)

Data/ML Logic for Non-Coders

Agenda

i. Repaso: Digital Business

ii. Repaso: IA

I. Analytics con LLM

II. Machine Learning con LLM

III. Programación con LLM

Epílogo. Segundo producto de datos del portafolio

i. Repaso: Digital Business

Cultura y liderazgo digital: Por qué son importantes

- La cultura impulsa el éxito de la evolución digital
- El análisis de datos mide el progreso de la transformación cultural
- Los *dashboards* de BI rastrean la eficacia del liderazgo
- Las métricas culturales predicen los resultados de la transformación
- La información en tiempo real permite tomar decisiones de liderazgo proactivas



Man holding microphone, by [CX Insight](#)

Cuatro pilares del éxito del liderazgo digital

- Mentalidad de transformación (curiosidad + creatividad)
- Cultura innovadora
- Agilidad organizativa
- Aprendizaje continuo

Midiendo la transformación cultural

- Puntuaciones de compromiso de los empleados
- Índices de adopción de herramientas digitales
- Colaboración entre departamentos
- Índices de éxito de la innovación
- Velocidad de comercialización
- Finalización de programas de aprendizaje
- Eficacia del liderazgo



El papel de la orquestación en la transformación cultural

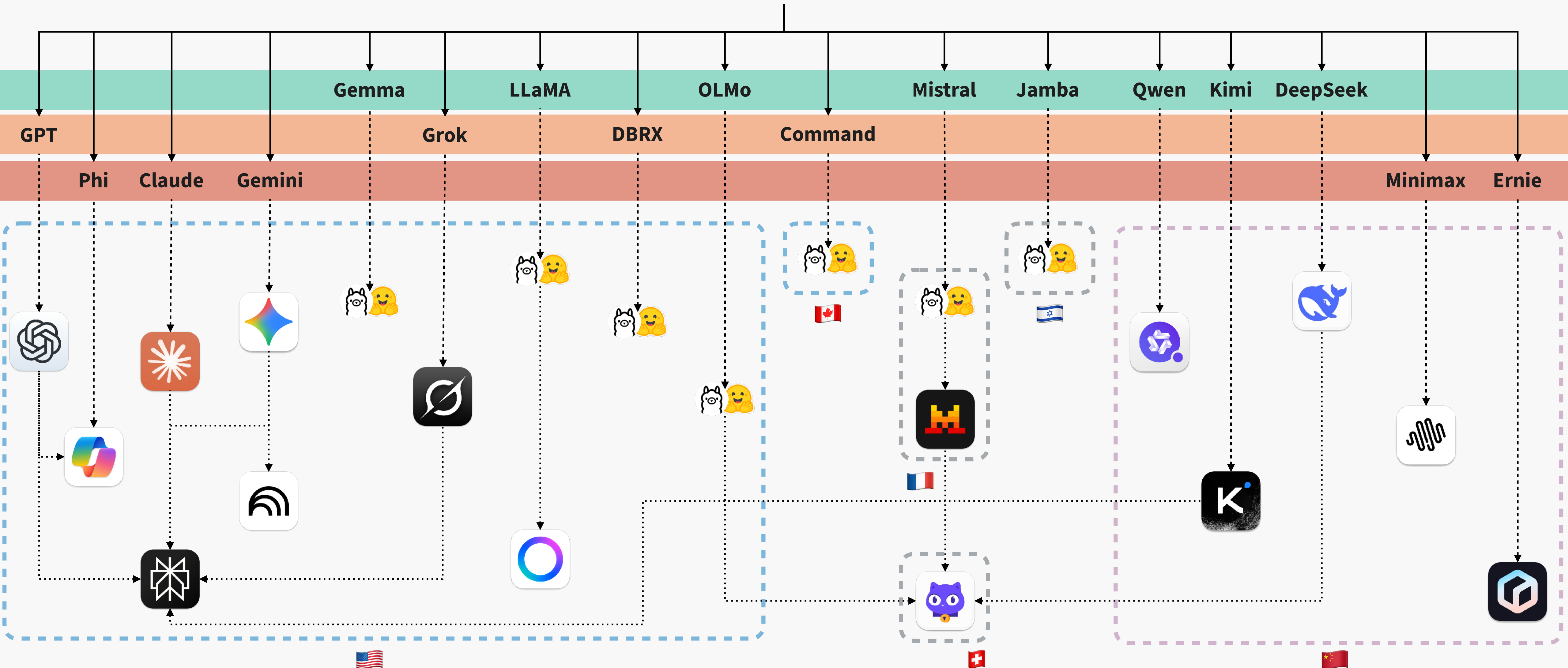
- Seguimiento de los KPI culturales
- Supervisar la eficacia del liderazgo
- Medir la adopción digital
- Analizar los patrones de colaboración
- Informar sobre el progreso de la transformación
- Identificar áreas de mejora

ii. Repaso: IA

📄 Universal Language Model Fine-tuning for Text Classification [ULMFiT]

📄 Attention is all you need

📄 Adaptive mixtures of local experts

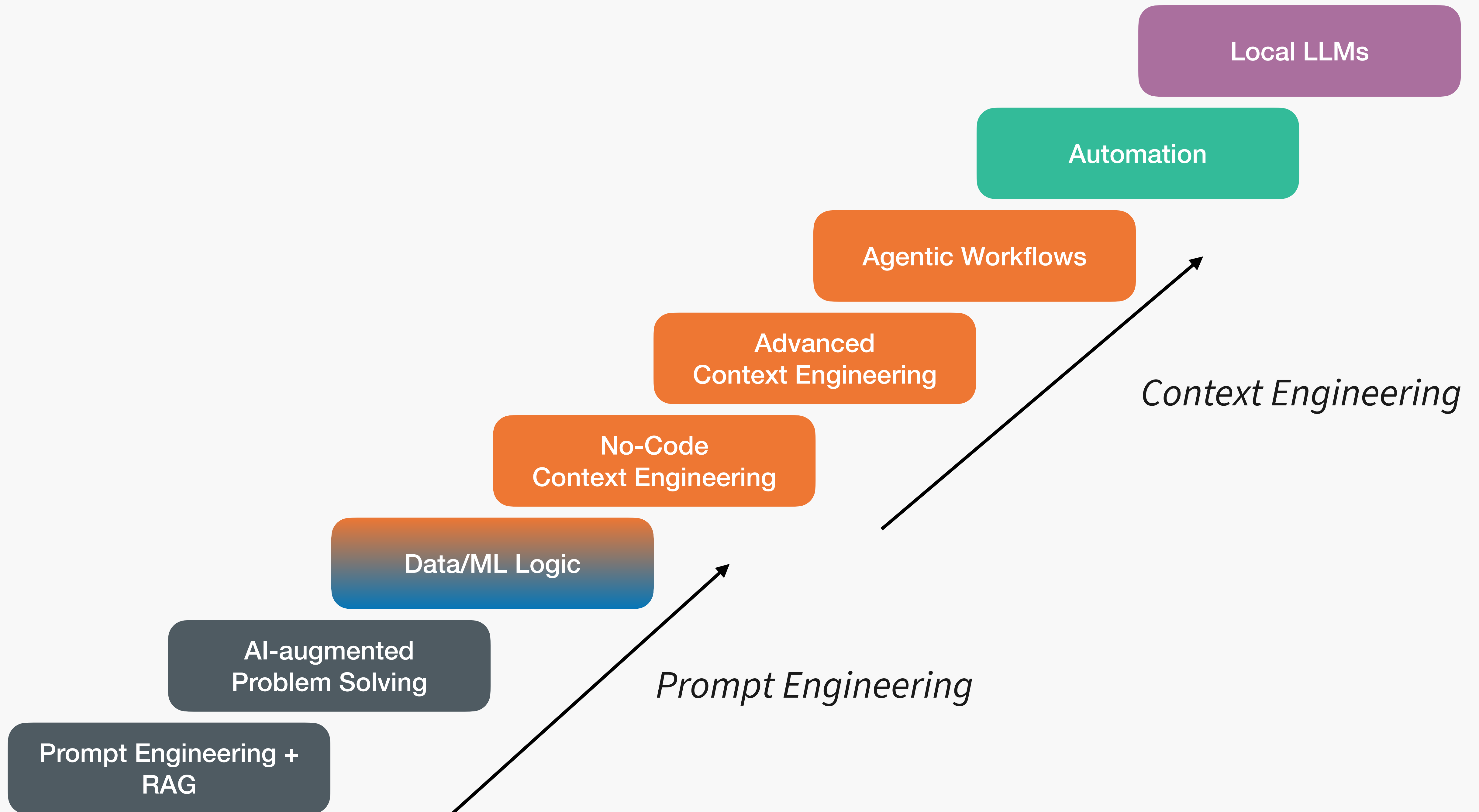


«¿Cuál es el mejor?»

—Pregunta típica

Usted elige en quién confiar.

No hay una aplicación más segura que todas las demás.



Una sintaxis de *prompt engineering*

Rol

«Actúa como...»

Tarea

«Vas a...»

Límites

«Sigue mis
instrucciones al
pie de la letra...»

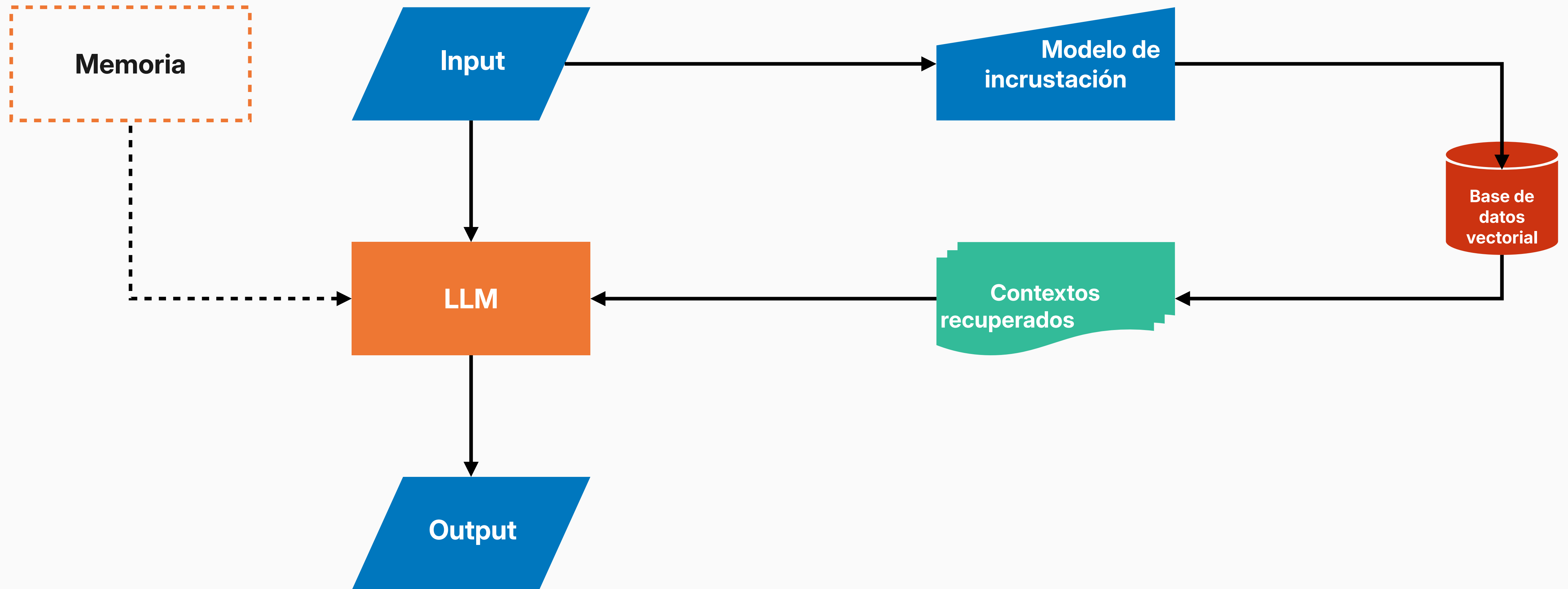
Recursión

«Explica paso a
paso tu
respuesta...»

Detalles

«X... Y... Z...»

RAG al rescate

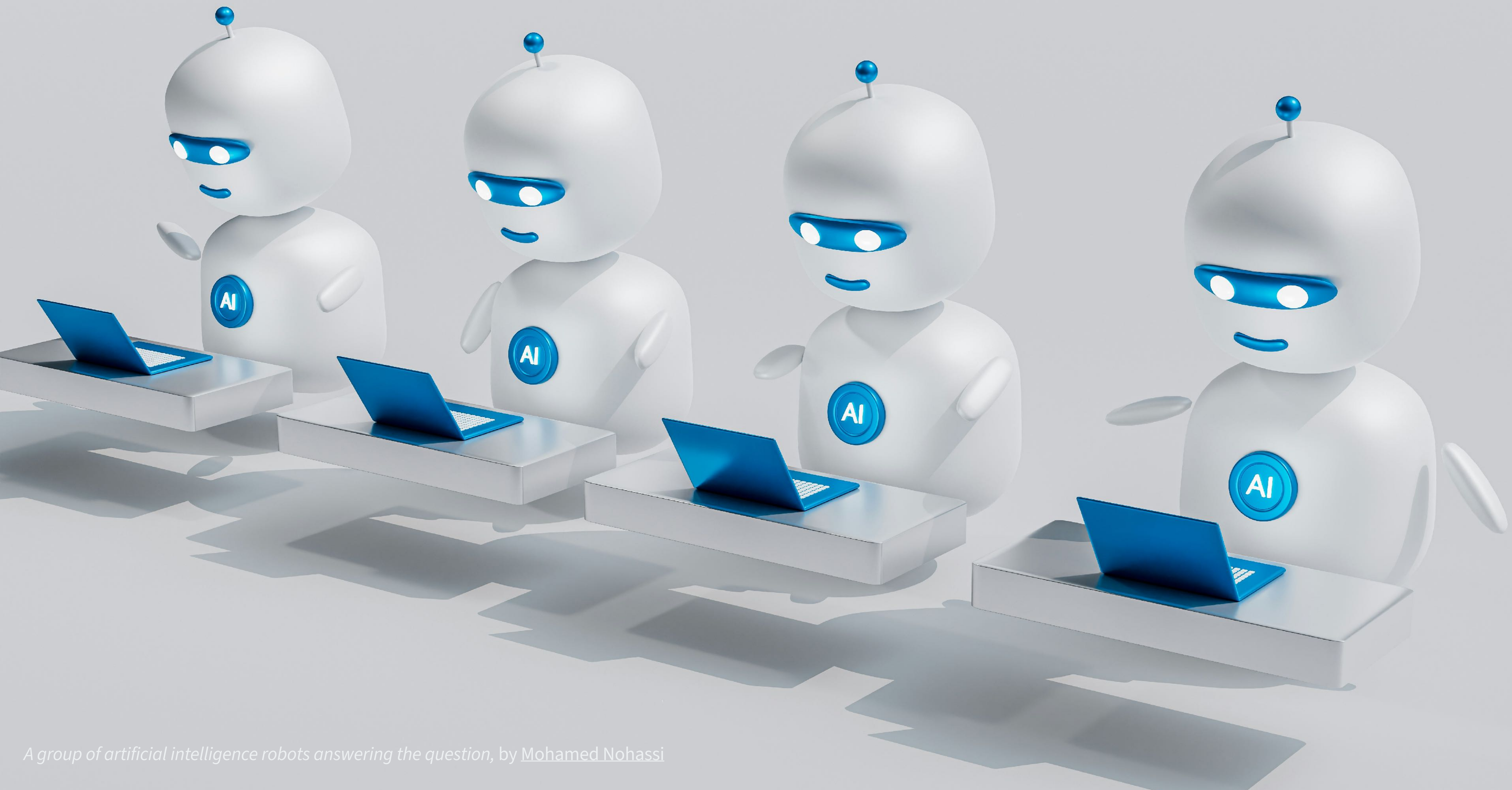




Woman playing chess, by [Michal Vrba](#)

El ejemplo muestra cómo CoT intenta «imitar» el razonamiento humano experto

- Desglosa las decisiones complejas en **componentes manejables**
- Sigue una **progresión lógica** desde los datos hasta las conclusiones
- Tiene en cuenta **múltiples factores** (costos, beneficios, riesgos)
- Hace **explícitas** las suposiciones implícitas
- Muestra los **pasos matemáticos/lógicos** que subyacen a las conclusiones cuantitativas
- Incluye tanto datos concretos como consideraciones **cualitativas**



A group of artificial intelligence robots answering the question, by Mohamed Nohassi

Self-Consistency para Smart KPI/OKR

- Panel de expertos
- CoT
- Consistencia → quién habla primero, en qué orden, sobre qué
- Consenso → Discrepancia → Recomendaciones finales

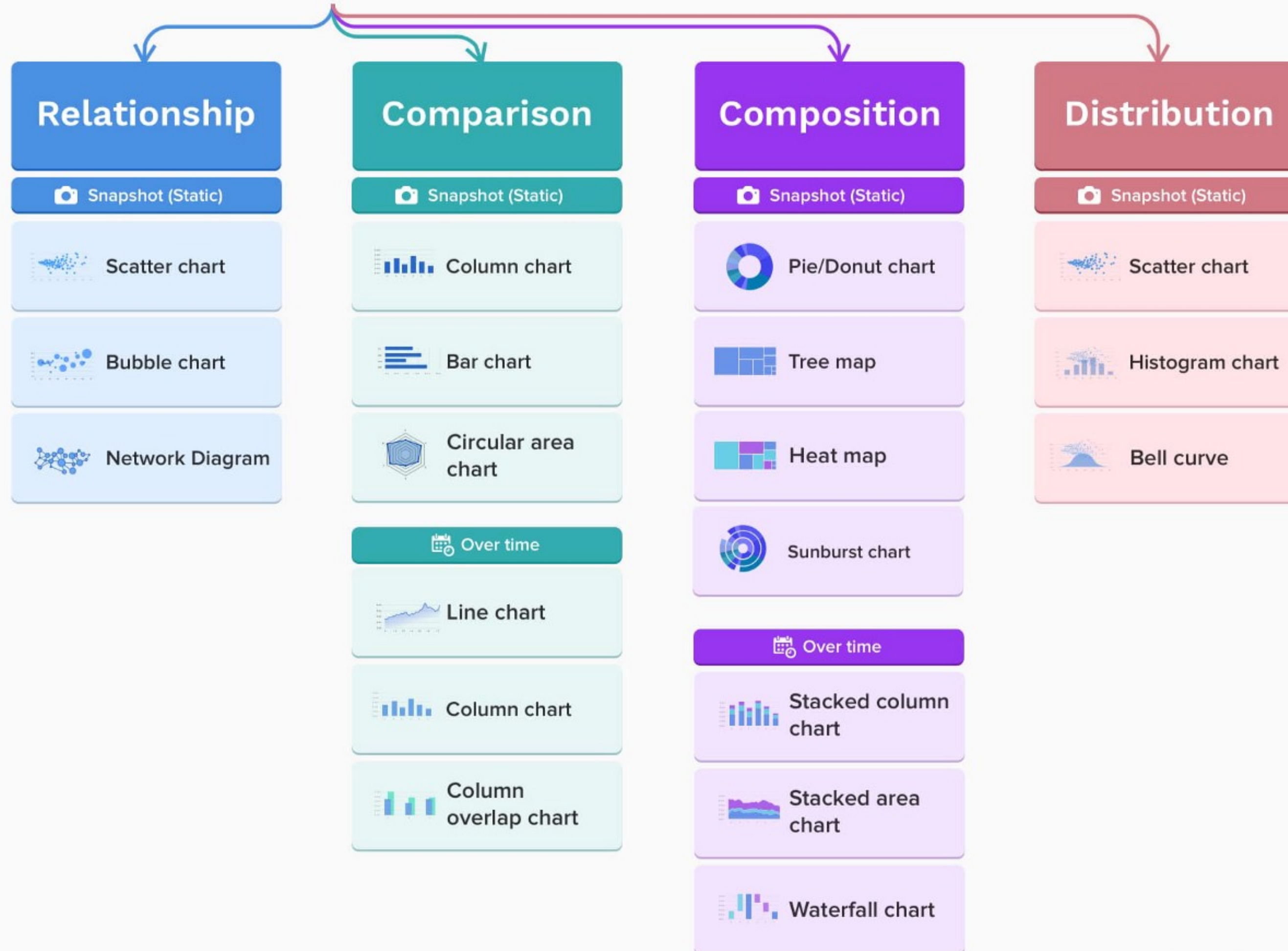
I. Analytics con LLM

Veamos primero 20 ideas para una
mejor visualización de datos

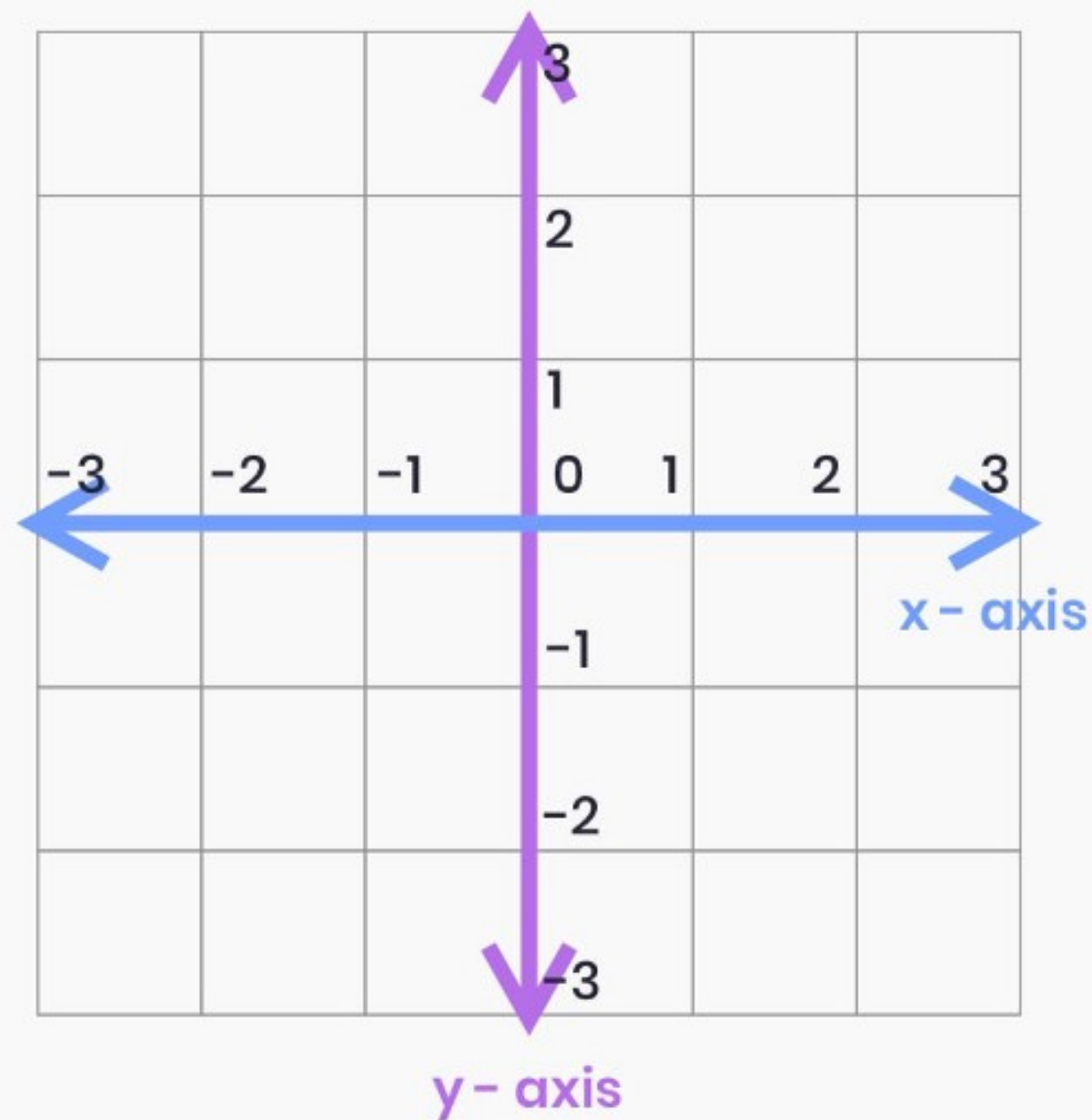
1. Elija el tipo de gráfico adecuado

👉 Choose the right chart

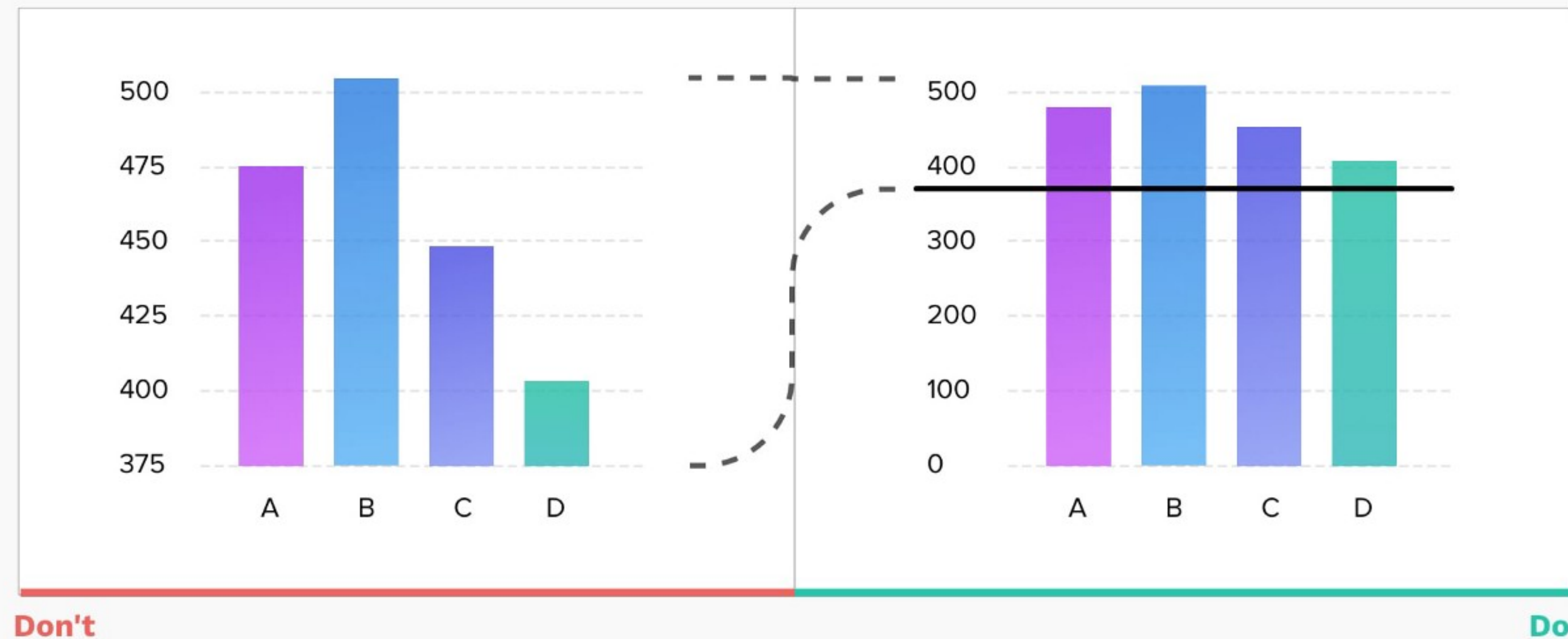
I would like to **see**



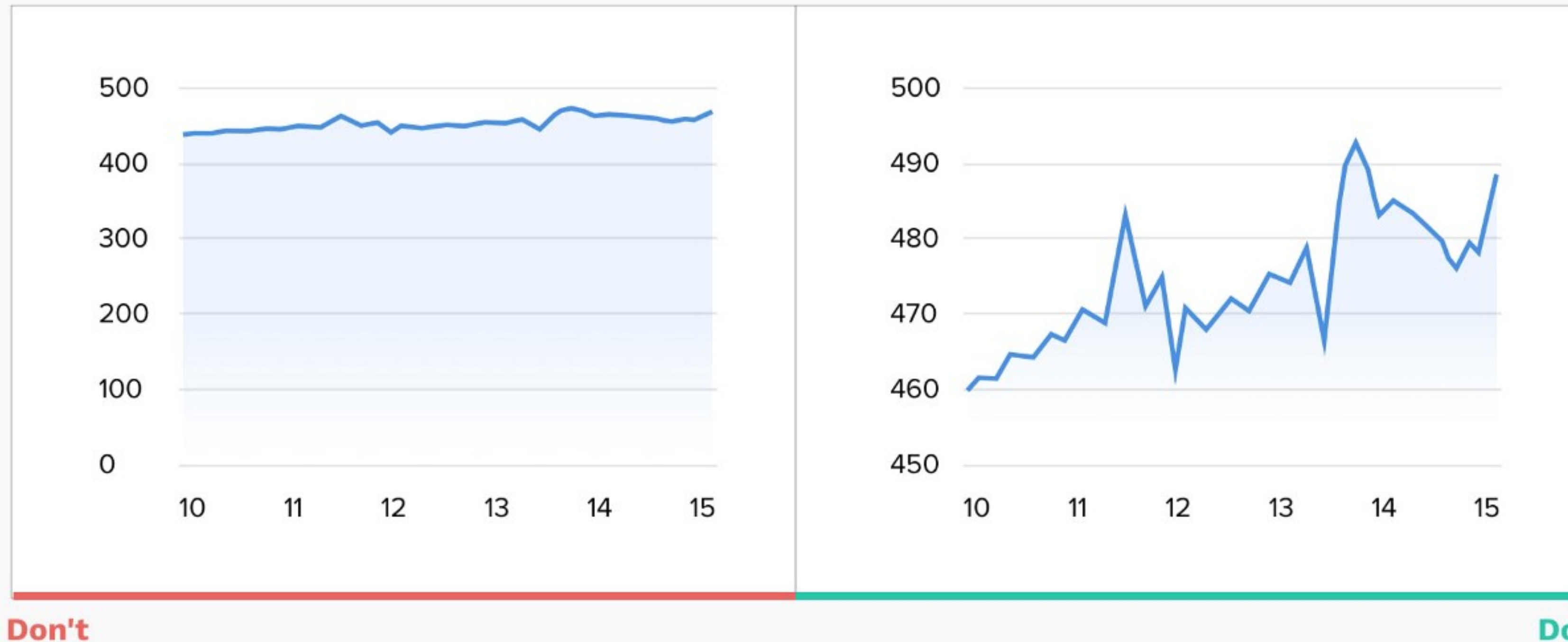
2. Use direcciones correctas basadas en los valores positivos y negativos



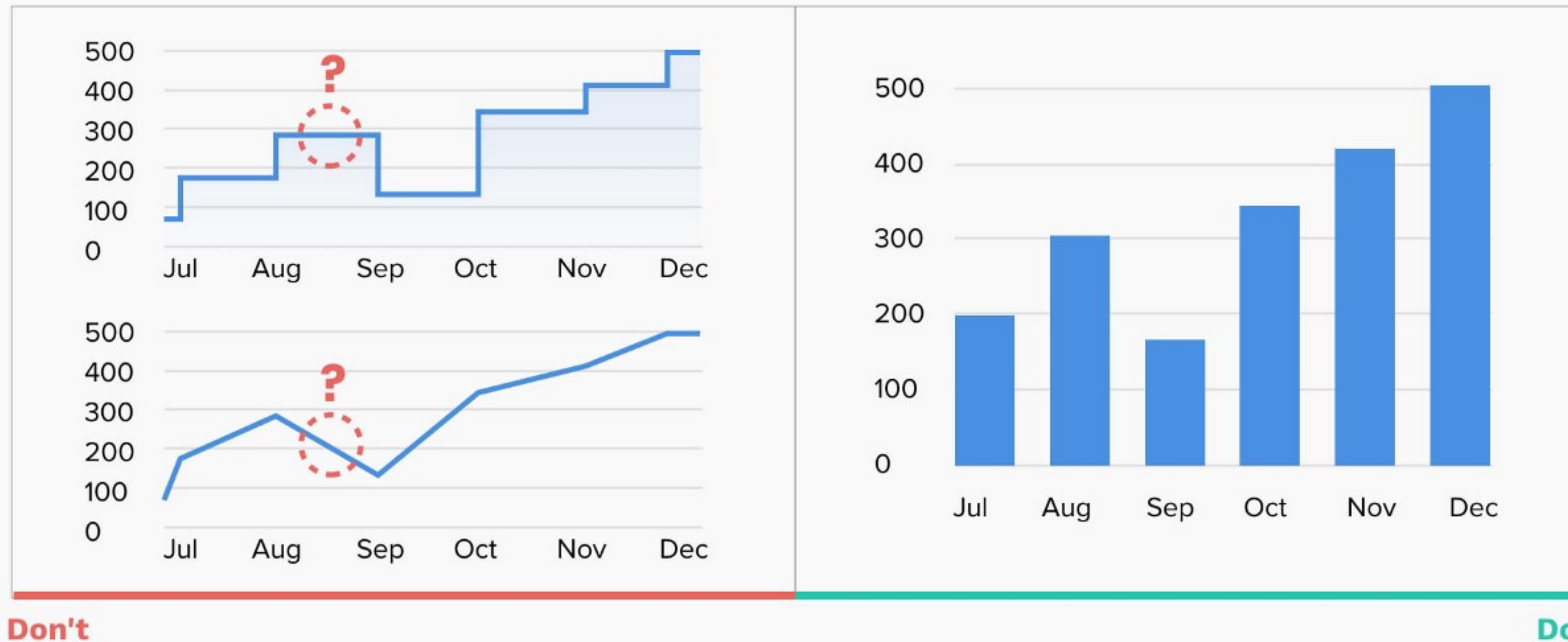
3. Comience el gráfico de barras siempre en la línea de base 0



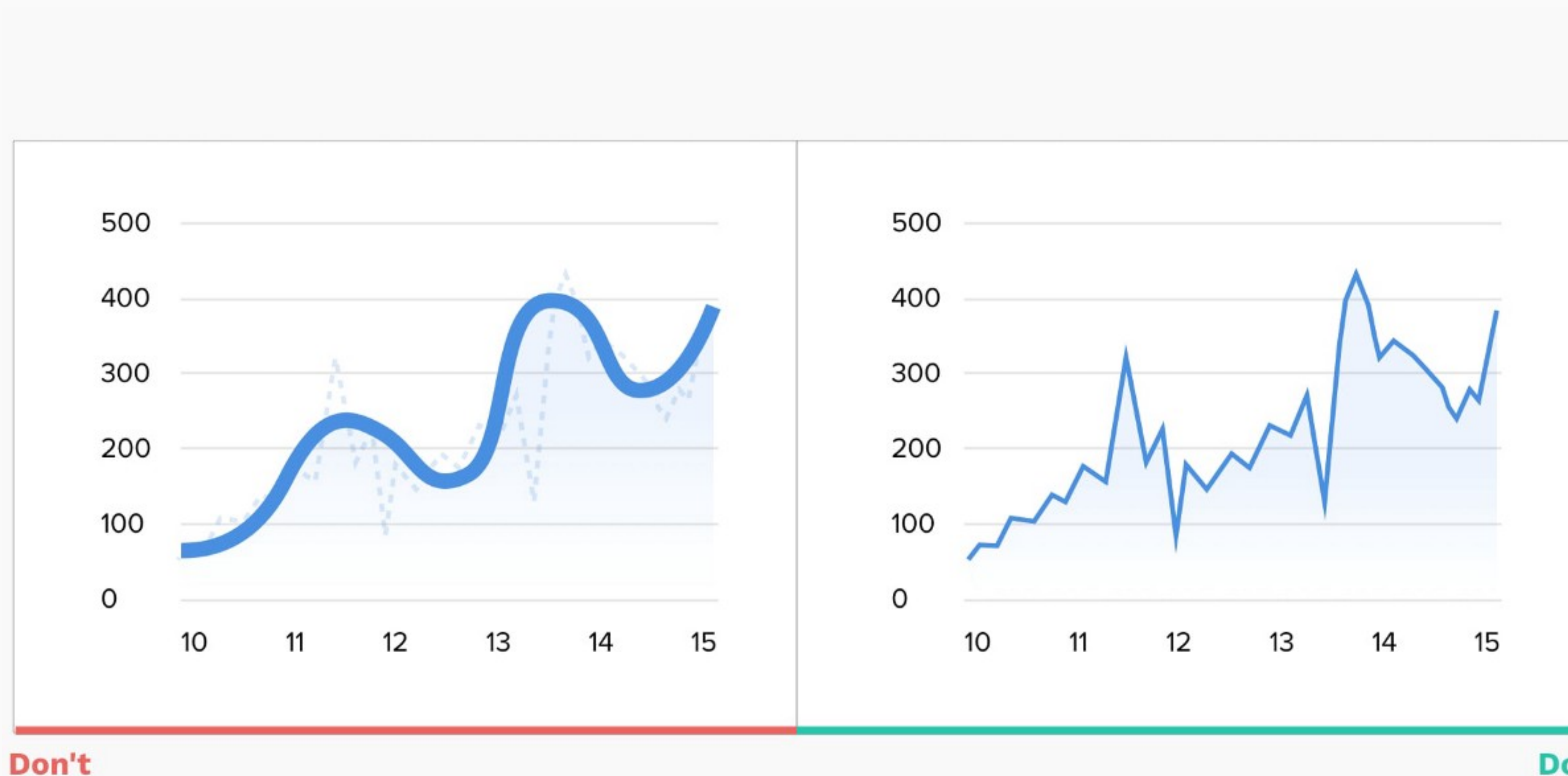
4. Use una escala adaptativa en el eje Y para los gráficos de líneas



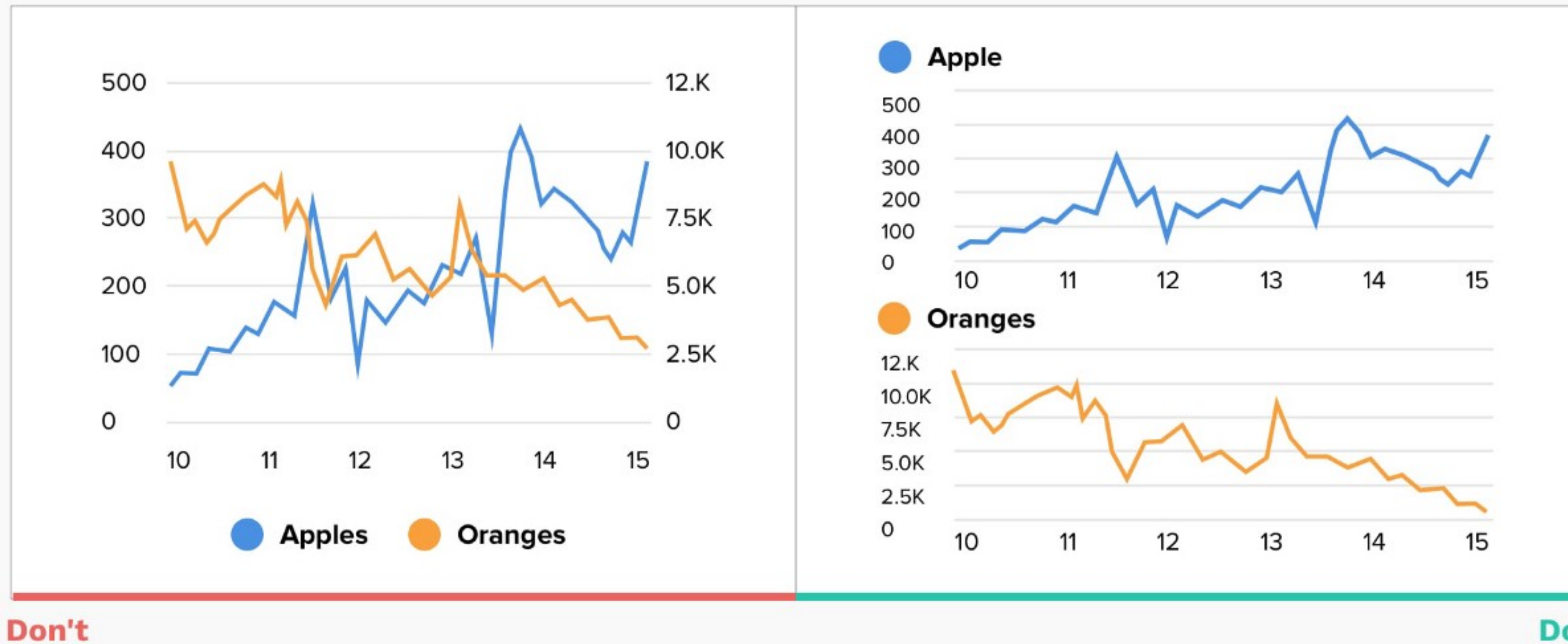
5. Tenga en cuenta las series temporales cuando utilice un gráfico de líneas



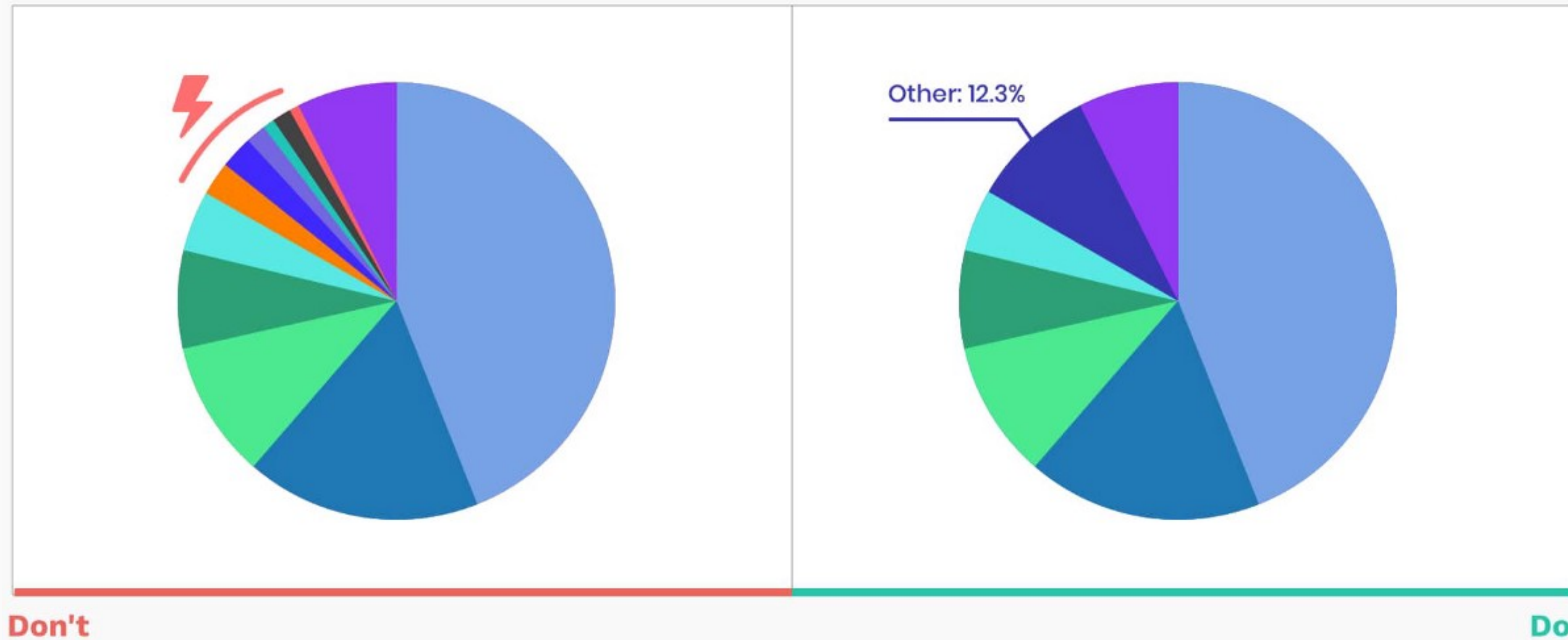
6. No use gráficos lineales «suavizados»



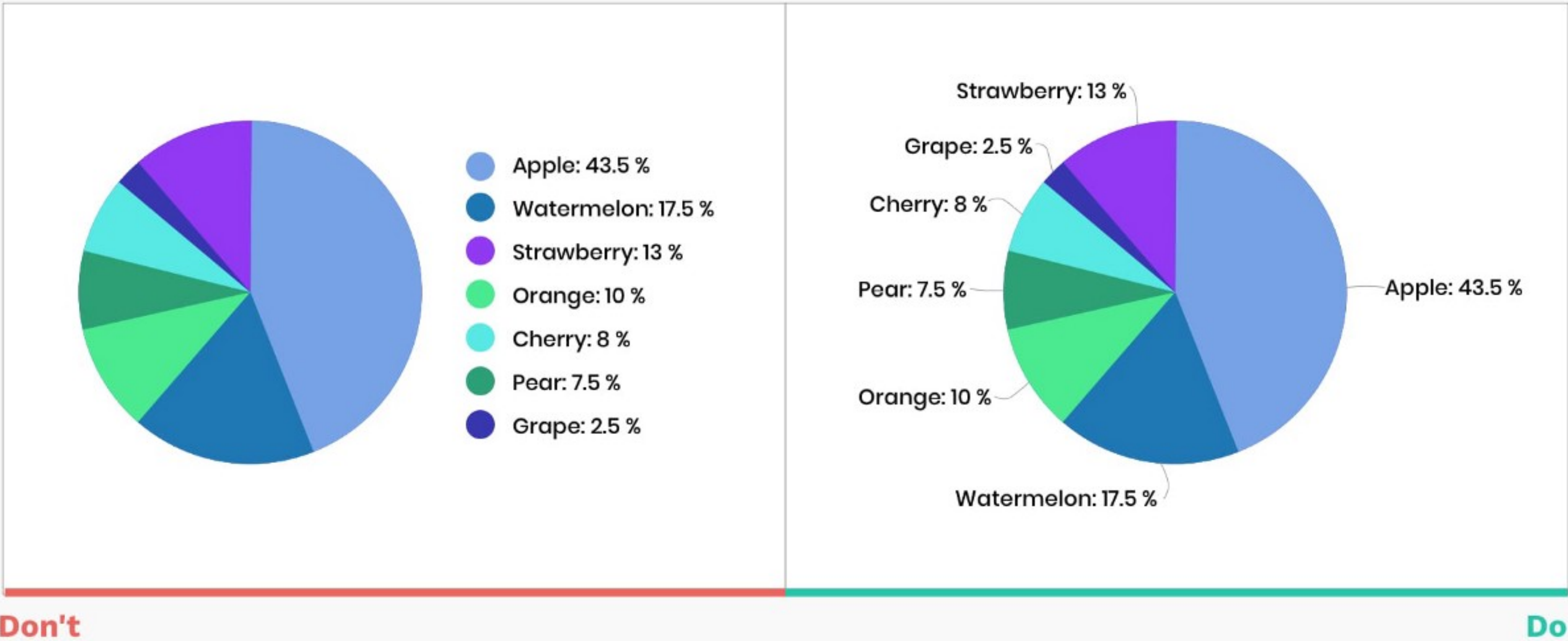
7. Evite el confuso doble eje



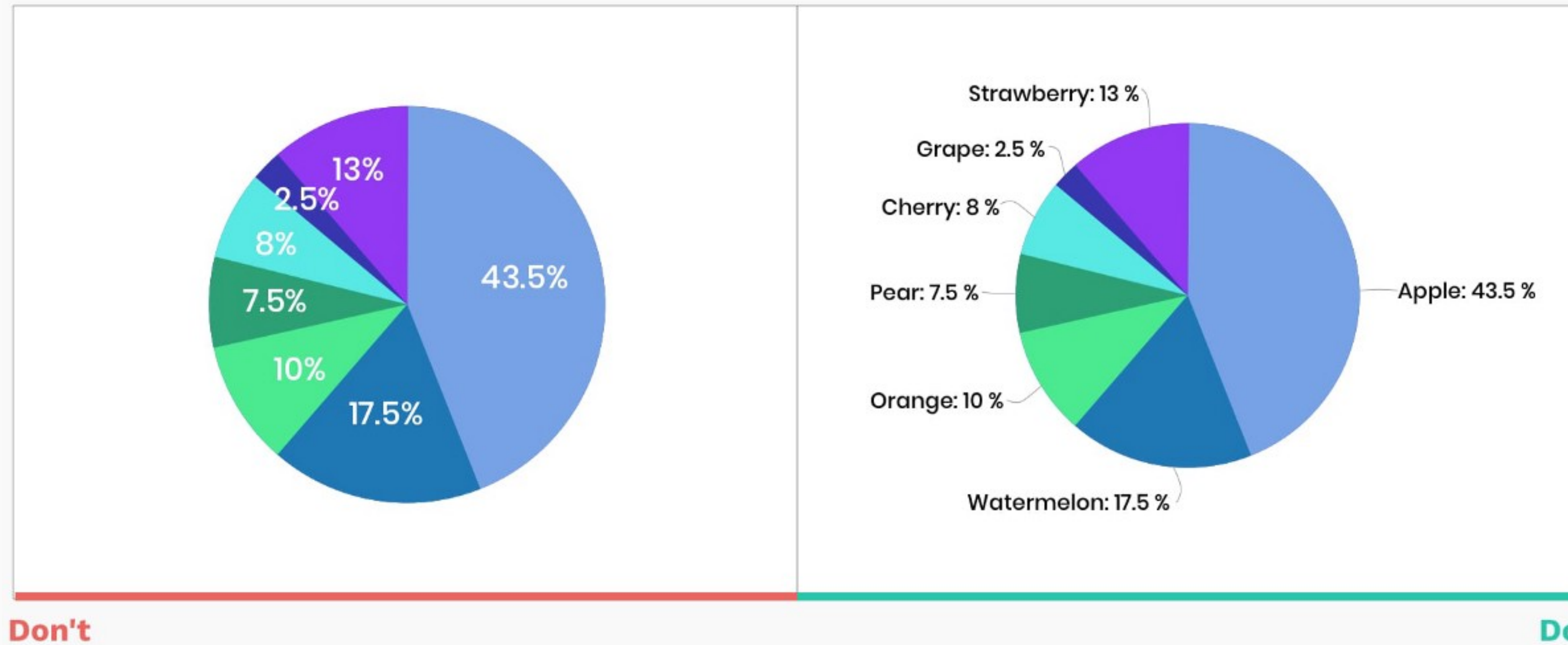
8. Limite el número de secciones mostradas en un gráfico circular



9. Etiquete directamente en el gráfico



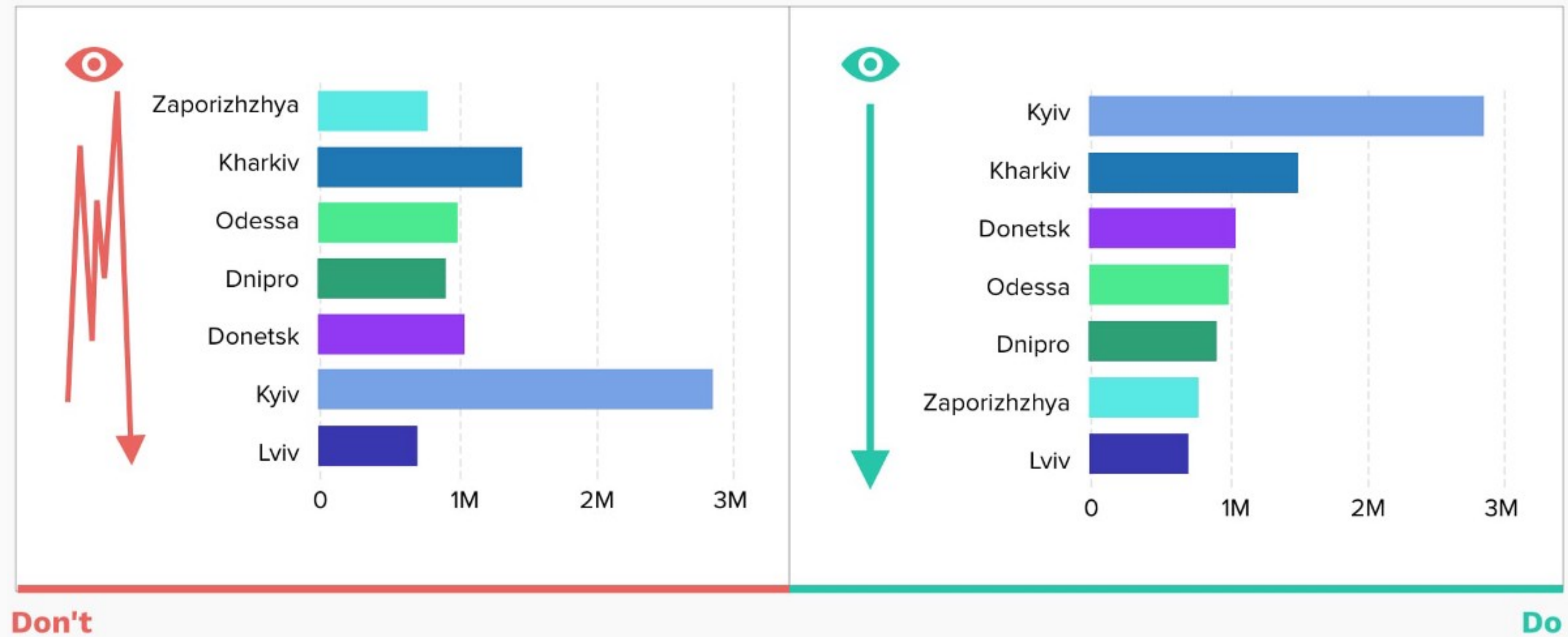
10. No etiquete encima de las secciones



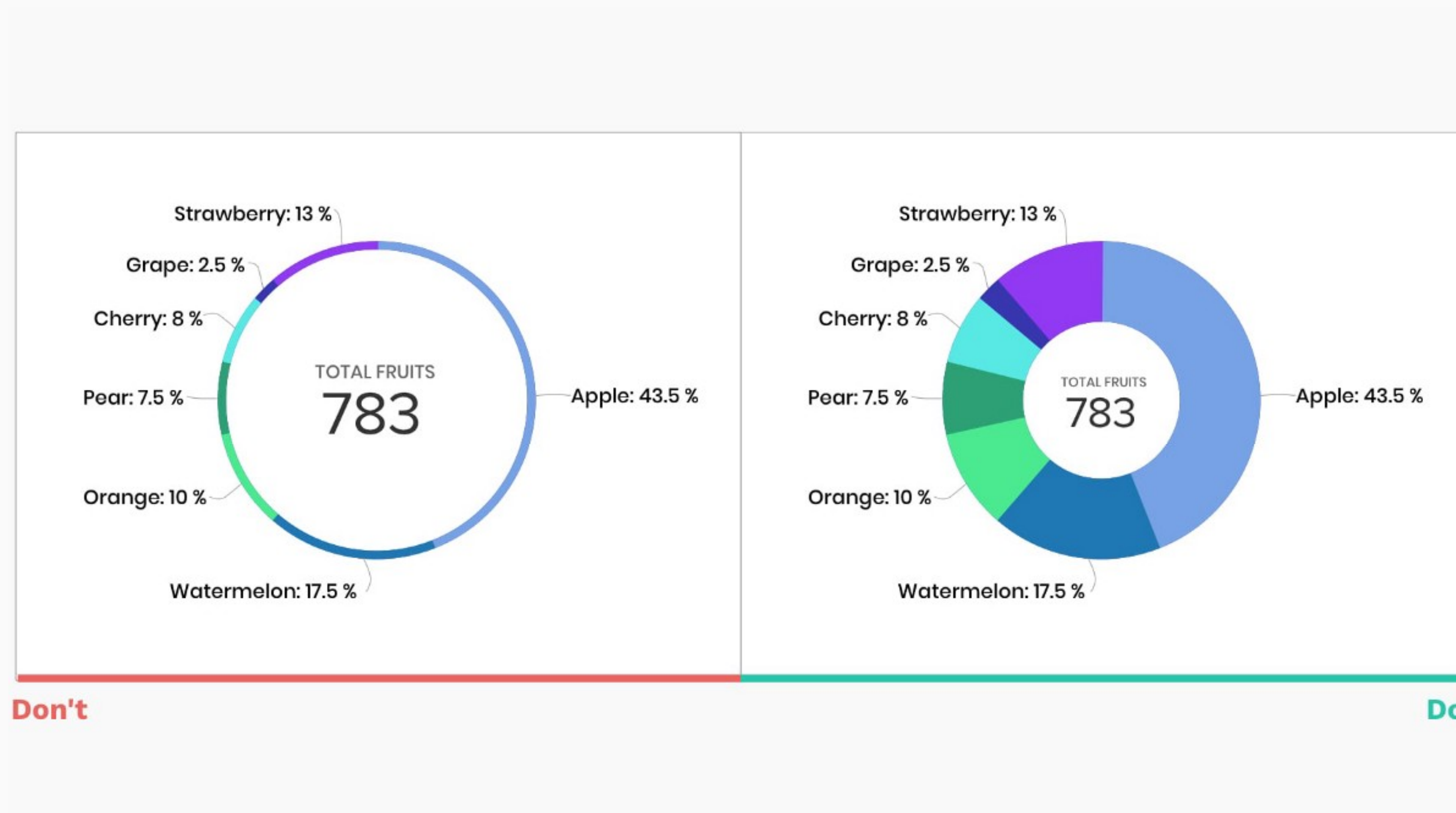
11. Ordene las secciones para un escaneo más rápido



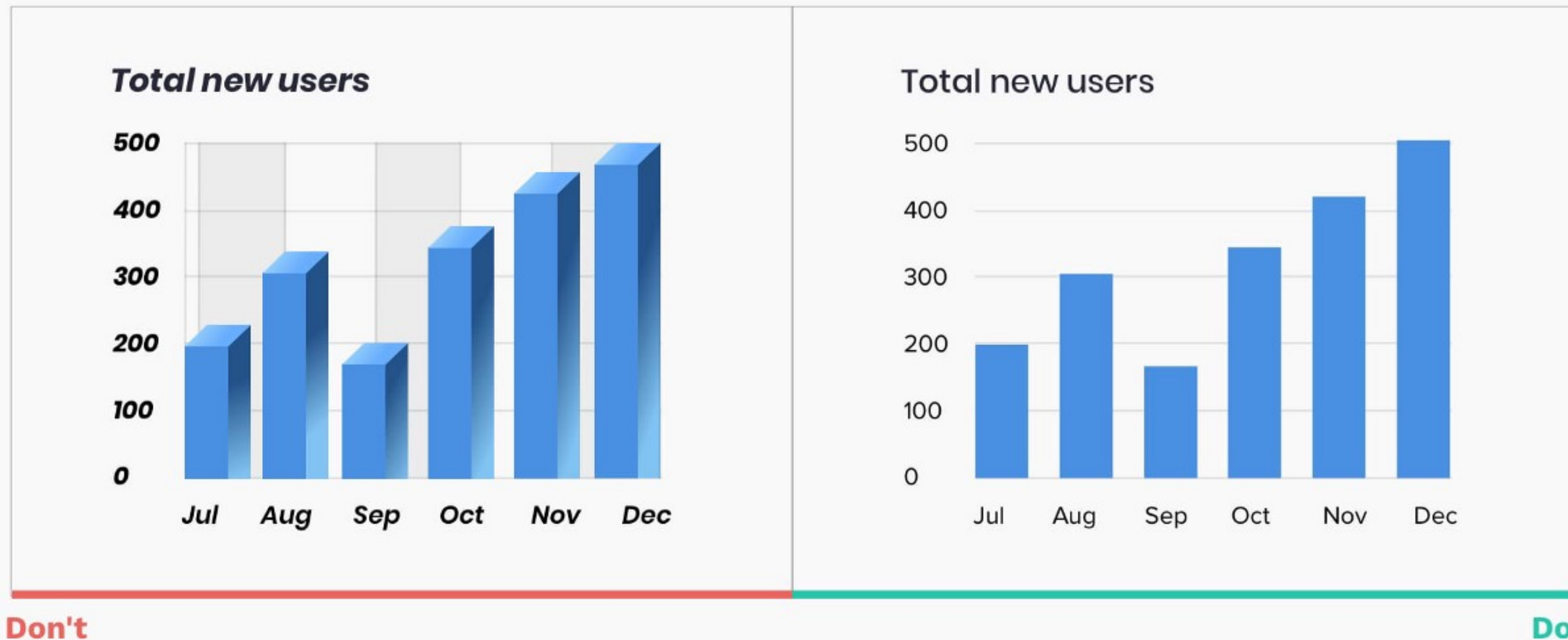
12. Evite la aleatoriedad



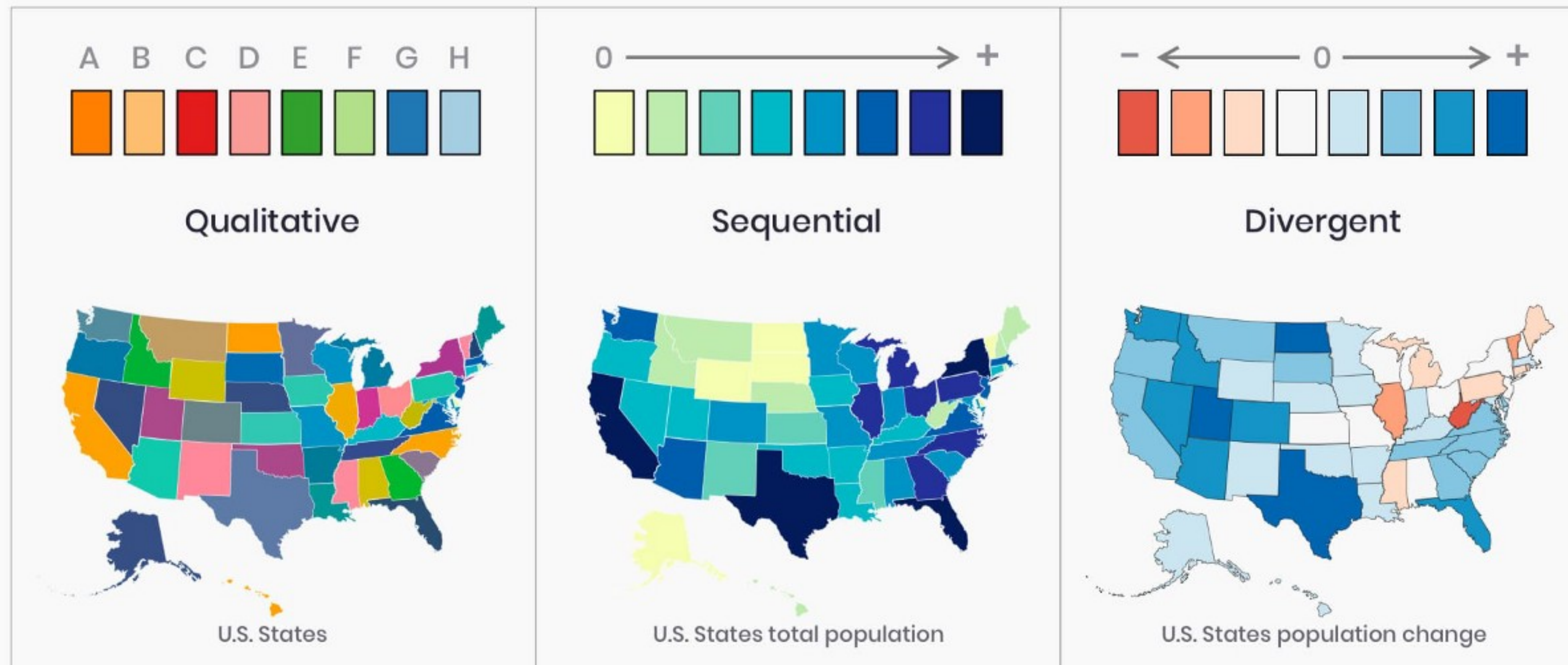
13. Los gráficos de anillo finos son imposibles de leer



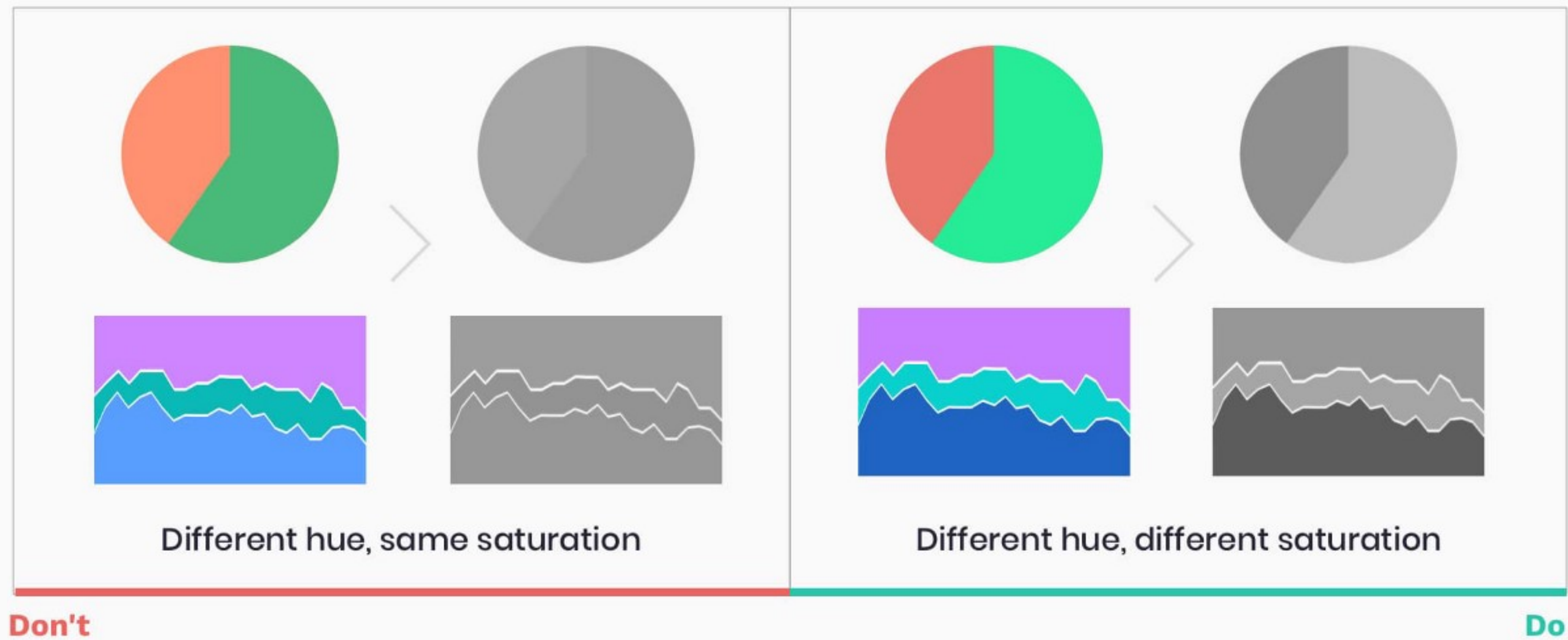
14. Deje que los datos hablen por sí mismos



15. Elija una paleta de colores que se ajuste a la naturaleza de sus datos



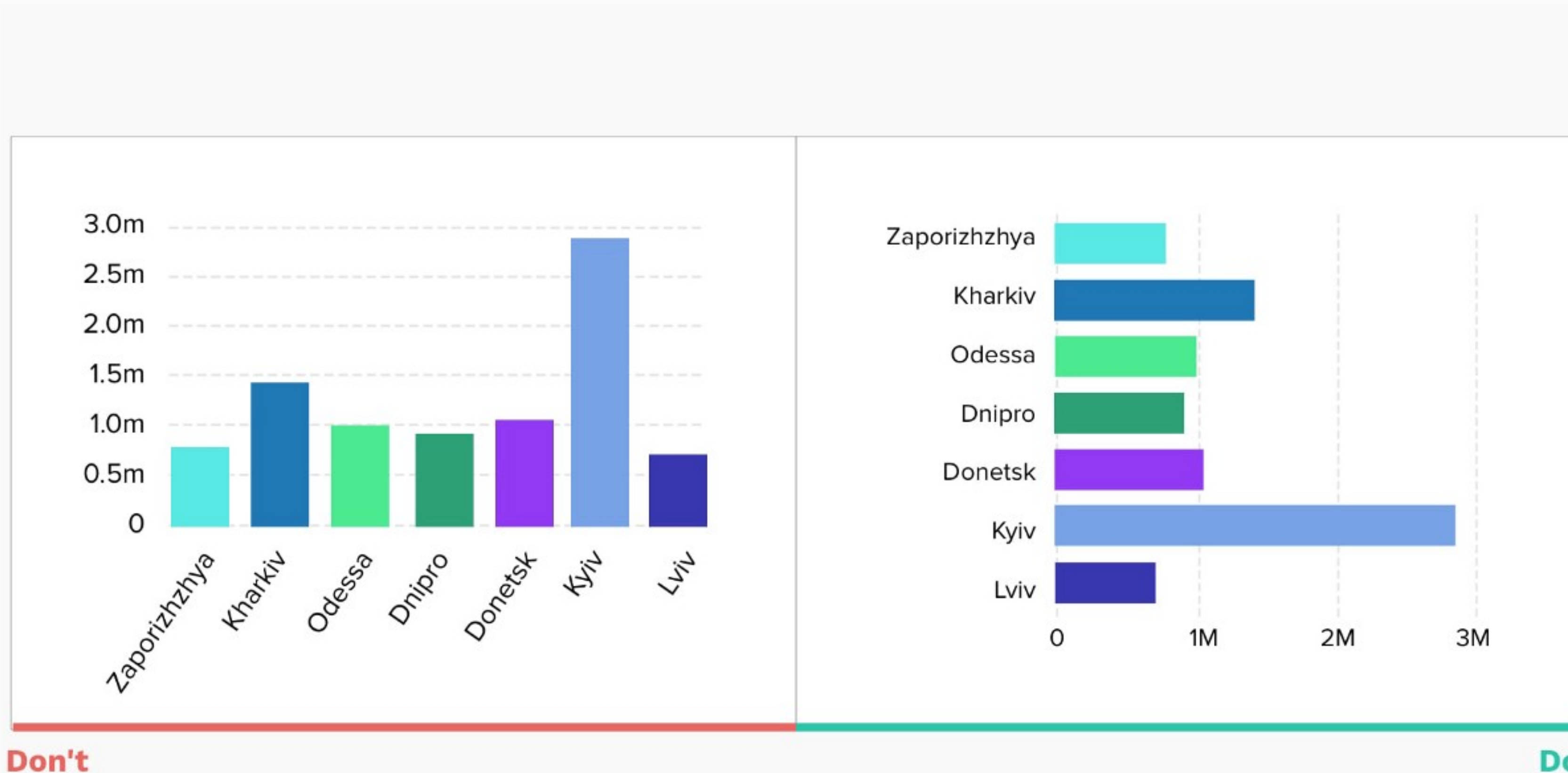
16. Diseño para la accesibilidad



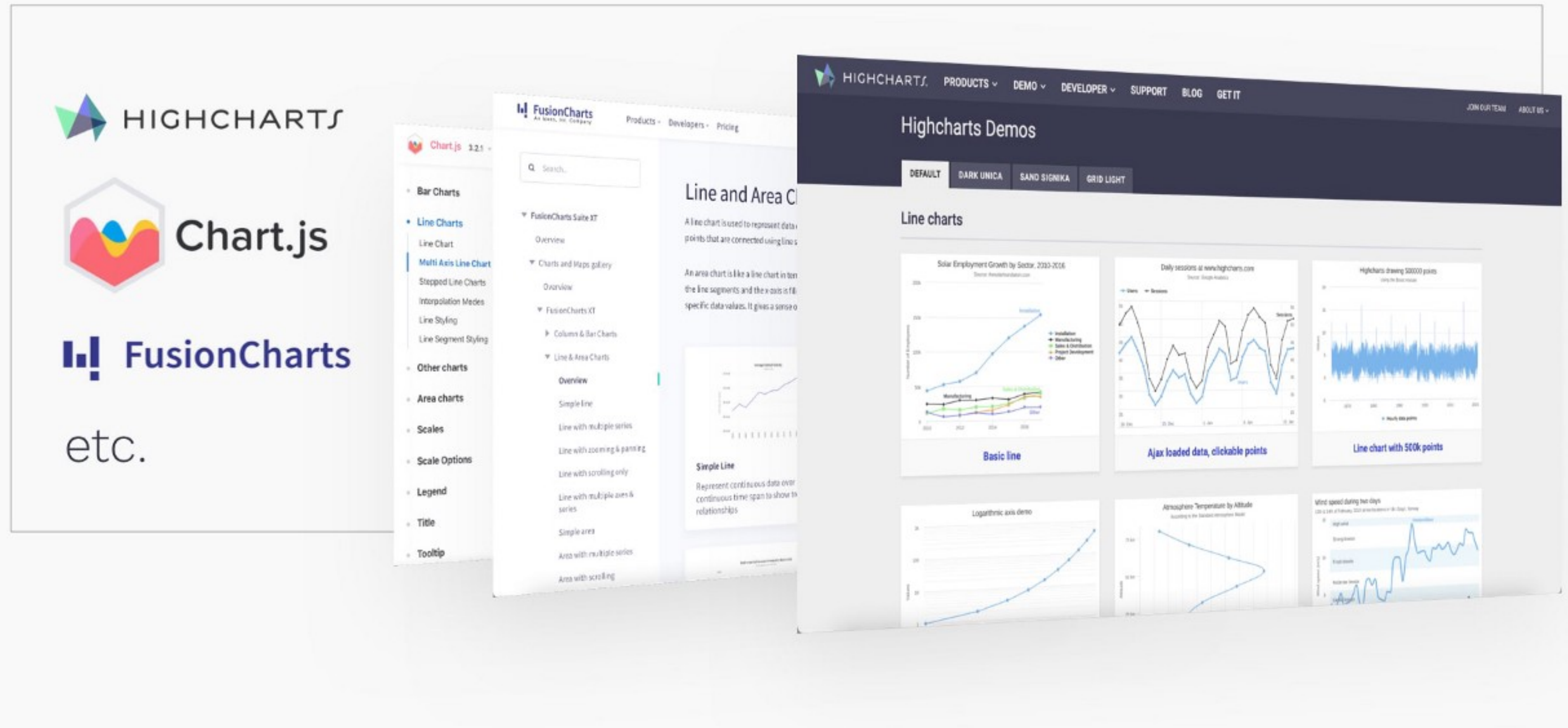
17. Enfóquese en la legibilidad



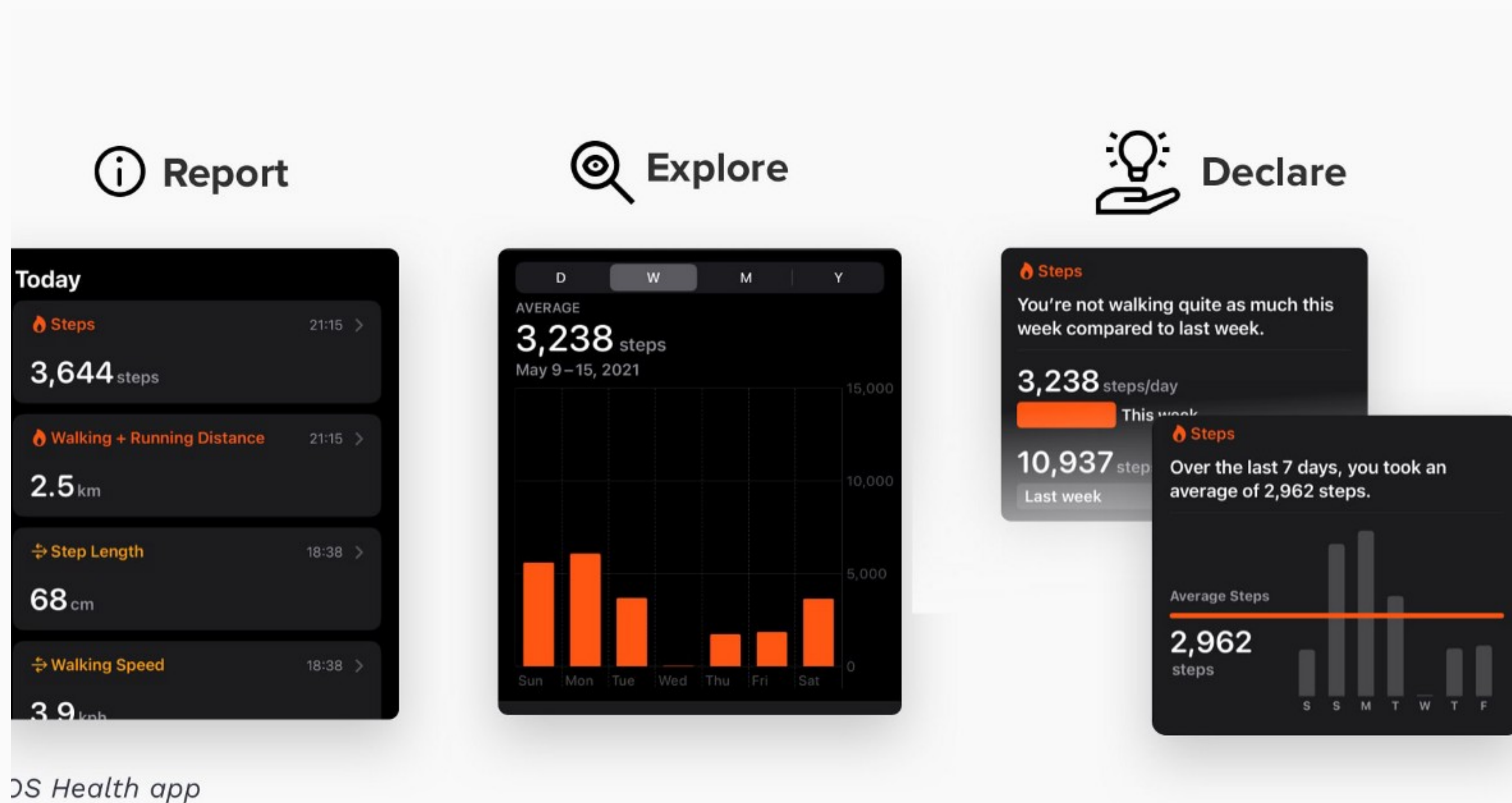
18. Use un gráfico de barras horizontales en lugar de etiquetas rotativas



19. Elija su biblioteca de gráficos



20. Vaya más allá de los informes estáticos



Es un modelo NLP: cuando queremos modificar la forma, estilo y colores de los gráficos, le hablamos como a un diseñador... siendo específicos pero sin usar palabras de más.

Idea clave

**Sin embargo, si podemos usar
conceptos «nativos» al modelo,
podemos personalizar la
visualización de forma más
eficaz.**

Trabajando con matplotlib y seaborn

- Paletas matplotlib
- Paletas seaborn

Trabajando con ggplot2

- Estilos ggplot2
- Más estilos ggplot2

Simulación de datos con LLM

Prompt detallado: tipo de datos que necesita, variables relevantes, intervalos de datos y distribución deseada

- Las **preguntas son clave**: Defina claramente la estructura de datos, los nombres de las variables y los patrones de datos deseados en su prompt para obtener resultados precisos
- Variedad de **tipos de datos**: Puede generar texto, números, fechas, horas, direcciones e incluso estructuras de datos complejas en función
- **Personalización**: Ajuste la cantidad de registros (filas), rangos de datos específicos e incluso la probabilidad de ciertos valores dentro del conjunto de datos generado
- **Aspectos estadísticos**: Distribución (representación realista) y «semilla» (reproducibilidad)

Ejemplos de *prompts*

- Datos básicos de clientes:
 - «Genera una lista de 100 registros de clientes con campos como 'Nombre', 'Correo electrónico', 'Número de teléfono', 'Ciudad' y 'Fecha de compra' con valores realistas.»
- Simulación de tráfico de sitios web:
 - «Crea un conjunto de datos de 1000 visitas a sitios web con 'Timestamp', 'Page URL', 'User ID', y 'Location' donde los tiempos de visita sigan una distribución normal.»
- Simulación de datos de sensores:
 - «Simula 24 horas de lecturas de sensores de temperatura con fluctuaciones aleatorias en torno a una temperatura base, incluyendo marcas de tiempo.»

Aunque el LLM puede generar datos realistas, es posible que no reproduzca a la perfección las complejidades del mundo real, por lo que siempre hay que revisar y validar los datos generados.

Idea clave

Para necesidades de simulación de datos más complejas, considere el uso de bibliotecas Python o R junto con un LLM para crear una lógica de generación de datos personalizada.

Para usuarios avanzados

Pipeline de análisis de datos

**No es nuestro foco: Inténtelo
luego con su propio conjunto de
datos.**

Además del preámbulo, debemos seguir una lógica de analítica de datos

EDA

¿Qué tenemos?

**Limpieza de
datos**

¿Qué hace falta
antes de iniciar
el análisis?

**Análisis
descriptivo**

¿Cómo luce
cada variable?

**Analítica
descriptiva**

¿Qué sucedió?

**Analítica
diagnóstica**

¿Por qué
sucedió?

Preámbulo

- Actúa como un analista de datos profesional. Tu trabajo consiste en analizar los conjuntos de datos que te proporcione y seguir mis instrucciones específicas. No te desvíes de mis instrucciones ni utilices información no incluida en los conjuntos de datos adjuntos. Cuando realices el análisis de datos, utiliza NumPy, SciPy, Pandas y otras bibliotecas de Python apropiadas para el análisis de datos. Debes titular cada gráfico que generes al estilo de una pregunta que se responde al mirar los gráficos. Por ejemplo, si el usuario desea graficar el crecimiento de los ingresos en los últimos cinco años fiscales para diferentes segmentos, un título apropiado podría ser «¿Qué segmento generó más efectivo en los últimos cinco años fiscales?» o algo similar. Después de cada gráfico, debes incluir un análisis breve pero detallado que explique las ideas principales o clave del gráfico basándose en el gráfico y en la información que he proporcionado. Luego, sugiere preguntas de seguimiento que haría un analista de datos para seguir encontrando información útil.

EDA (Análisis exploratorio de datos)

- **Algunas instrucciones básicas**

- *Cuéntame más sobre este conjunto de datos. Descríbeme brevemente cada columna.*
- *Realiza estadísticas descriptivas de cada columna. Para ello, agrupa las columnas numéricas y no numéricas en tablas diferentes, con cada columna como una fila.*
- *Realiza un análisis exploratorio de datos en cada una de estas columnas. Proporciona una visualización adecuada para representar el contenido de cada columna. Por ejemplo, utiliza un histograma para las columnas numéricas.*
- *(Profundiza en más columnas.)*
- *(Continúa con las columnas restantes.)*

Limpieza de datos

- **Dos necesidades comunes**

- *En la columna de [columna], parece que algunos valores tienen espacios innecesarios. Elimina estos espacios para clasificar mejor los datos.*
- *Limpia la columna [columna] eliminando la palabra [palabra innecesaria] de cada categoría y renombrando esta columna a [nuevo nombre].*

Análisis descriptivo

- **Varias formas de describir los datos**

- *Grafica las columnas de [variable] para [frecuencia 1], [frecuencia 2] y [frecuencia 3].*
- *Grafica las 10 principales [categoría] en función de [variable de interés].*
- *Grafica las 10 [categoría] más comunes que incluyen datos con [frecuencia X]. Representalo como un gráfico de barras para [variable].*
- *(Grafica los 10 [categoría] más comunes que incluyan datos de [frecuencia Y]. Representalo como un gráfico de barras para [variable]).*
- *Analiza la evolución de la [variable] mes a mes mediante un gráfico de líneas.*
- *Compara [variable X] con [variable Y] mediante un diagrama de dispersión. Incluye una línea de tendencia utilizando un modelo lineal y un tipo de línea de puntos.*

Analítica descriptiva

- «¿Qué sucedió?»
 - *Haz un gráfico lineal utilizando el valor mediano de la columna [columna Y] y la columna [columna X] como eje X. Agrega estos datos de forma mensual. No incluyas los datos de [mes] de [año], ya que este conjunto de datos no tiene una colección completa de datos para ese mes.*
 - *Haz un gráfico similar utilizando un gráfico de barras que muestre el número de [variable X] a partir de los datos de [variable Y]. Asegúrate de que esta vez el eje X está ordenado por mes, no de mayor a menor.*

Analítica diagnóstica

- «¿Por qué sucedió?»

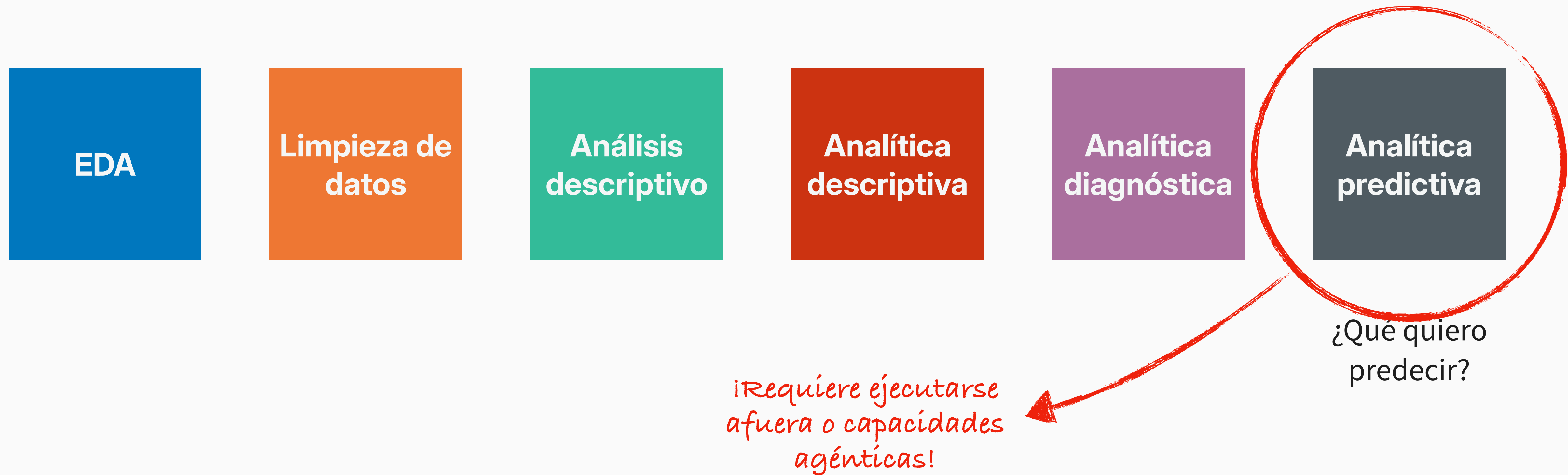
- *Utilizando un gráfico lineal, haz un gráfico similar al anterior utilizando la columna [columna]. Para ello, utiliza el recuento de los 5 [categoría] más comunes. Para este recuento, asegúrate de que se trata sólo de [categoría] que tienen datos de [frecuencia].*
- *(Proporciona una tabla de los 5 principales [categoría] en abril que muestre el título de [categoría], [variable X] y [variable Y]).*
- *(Desglosa aún más esta tabla mostrando esto pero con [categoría Y] también en el [categoría X]).*
- *Proporciona histogramas similares a los anteriores, pero represéntalos en el mismo gráfico para [mes 1], [mes 2] y [mes 3] de [año]. Represéntalos en gráficos separados pero dentro del mismo visual: no los quiero superpuestos.*

II. Machine Learning con LLM

Sintaxis
+ Personalización
+ Especialización
+ Sofisticación

Lenguaje de la GenAI

¿Qué pasa si quiero predecir algo?



Tres ejemplos de ML

Churn analysis

- Actúa como una científica de datos experta en modelos de predicción que utilizan machine learning. Tu tarea es analizar los conjuntos de datos que te adjunte y seguir mis instrucciones específicas. No te desvíes de mis instrucciones ni utilices información no incluida en los conjuntos de datos adjuntos. Cuando realices los modelos de predicción, utiliza NumPy, SciPy, Pandas y otras bibliotecas de Python apropiadas para este objetivo. Empecemos: Utilizando mi conjunto de datos sobre rotación de clientes de tecnologías de la información y la comunicación (TIC), ayúdame a escribir código Python para leer detalladamente el conjunto completo de los datos y describirlos. Asegúrate de incluir comentarios en el código que me ayuden a entender qué hace cada línea importante de código. Solo debes escribir el código, no ejecutarlo.
- Ahora, conserva el código anterior y agrega las líneas de código necesarias para entrenar (train) un modelo de regresión logística, usando la semilla seed=1234 para cualquier elemento aleatorio. El código debe usar exactamente el 80% de los datos para entrenar el modelo (train) y exactamente 20% para probarlo (test). Luego, agrega el código necesario para probar (test) el modelo y mostrar sus coeficientes. El código debe guardar los resultados de los coeficientes en una tabla en un archivo CSV. Solo debes escribir el código, no ejecutarlo.
- Ahora, conserva el código anterior y agrega las líneas de código necesarias para crear visualizaciones de:
 - El límite de decisión (decision boundary)
 - Distribuciones de probabilidad para clientes que abandonan (churners) y clientes que no abandonan (non-churners)
 - Gráfico de importancia de las características (feature importance)El código debe guardar los resultados gráficos en archivos PNG. Solo debes escribir el código, no ejecutarlo.
- Ahora, usando los archivos adjuntos, explica qué significan los coeficientes del modelo para la pérdida de clientes. Explica también los resultados gráficos: ¿Son apropiados para acompañar el resultado numérico? ¿Qué más sugieres hacer con las visualizaciones si deseo mantener el modelo de regresión logística? Explica los resultados globales en términos empresariales.

Análisis de riesgo de impago

- Actúa como una científica de datos experta en modelos de predicción que utilizan machine learning. Tu tarea es analizar los conjuntos de datos que te adjunte y seguir mis instrucciones específicas. No te desvíes de mis instrucciones ni utilices información no incluida en los conjuntos de datos adjuntos. Cuando realices los modelos de predicción, utiliza NumPy, SciPy, Pandas y otras bibliotecas de Python apropiadas para este objetivo. Empecemos: Utilizando mi conjunto de datos de evaluación del riesgo de impago de las tarjetas de crédito, ayúdame a escribir código Python para leer detalladamente el conjunto completo de los datos y describirlos. Asegúrate de incluir comentarios en el código que me ayuden a entender qué hace cada línea importante de código. Solo debes escribir el código, no ejecutarlo.
- Ahora, conserva el código anterior y agrega las líneas de código necesarias para entrenar (train) un modelo de árbol de decisión usando la semilla seed=1234 para cualquier elemento aleatorio. El código debe usar exactamente el 80% de los datos para entrenar el modelo (train) y exactamente 20% para probarlo (test). El código debe utilizar un max_depth que sea estadísticamente óptimo pero al mismo tiempo apropiado para optimizar la claridad de las visualizaciones. Luego, agrega el código necesario para probar (test) el modelo y mostrar sus coeficientes. El código debe guardar los resultados del modelo en una tabla en un archivo CSV. Solo debes escribir el código, no ejecutarlo.
- Ahora, conserva el código anterior y agrega las líneas de código necesarias para crear una visualización interpretable del árbol. Evita usar nombres complicados para las ramas y asegúrate de que los nombres de los features caben bien en su recuadro. El código debe guardar los resultados gráficos en archivos PNG. Solo debes escribir el código, no ejecutarlo.
- Ahora, usando los archivos adjuntos, muestra las reglas de decisión más importantes y explica su impacto en el negocio. Identifica los patrones clave en la evaluación del riesgo de impago de las tarjetas de crédito. Muestra cómo los distintos perfiles de cliente siguen caminos diferentes en el árbol.

Análisis de segmentación

- Actúa como una científica de datos experta en modelos de predicción que utilizan machine learning. Tu tarea es analizar los conjuntos de datos que te adjunte y seguir mis instrucciones específicas. No te desvíes de mis instrucciones ni utilices información no incluida en los conjuntos de datos adjuntos. Cuando realices los modelos de predicción, utiliza NumPy, SciPy, Pandas y otras bibliotecas de Python apropiadas para este objetivo. Empecemos: Utilizando mi conjunto de datos sobre el comportamiento de compra de los clientes, realiza la preparación de datos necesaria: elimina las columnas innecesarias; estandariza todas las características necesarias; utiliza StandardScaler de sklearn para la normalización. Asegúrate de incluir comentarios en el código que me ayuden a entender qué hace cada línea importante de código. Solo debes escribir el código, no ejecutarlo.
- Ahora, conserva el código anterior y agrega las líneas de código necesarias para entrenar (train) un modelo de k-means clustering usando la semilla seed=1234 para cualquier elemento aleatorio. El código debe usar exactamente el 80% de los datos para entrenar el modelo (train) y exactamente 20% para probarlo (test). El código debe utilizar un valor k apropiado para optimizar la visualización que también sea estadísticamente adecuado. Luego, agrega el código necesario para probar (test) el modelo y mostrar sus coeficientes. El código debe guardar los resultados del modelo en una tabla en un archivo CSV. Solo debes escribir el código, no ejecutarlo.
- Ahora, conserva el código anterior y agrega las líneas de código necesarias para crear visualizaciones que muestren: Distribuciones de clusters; Posiciones del centroide; Características clave de cada cluster. El código debe guardar los resultados gráficos en archivos PNG. Solo debes escribir el código, no ejecutarlo.
- Ahora, usando los archivos adjuntos, genera perfiles detallados para cada segmento de clientes describiendo sus características clave: Nivel de ingresos; Patrones de gasto por categorías; Comportamiento de compra (total purchases, online proportion, deal propensity); Composición familiar (niños/adolescentes); Actividad reciente (recency). Finalmente, explica las implicaciones comerciales de estos segmentos.

Errores comunes y cómo solventarlos

- La cuestión de la reproducibilidad
 - Los LLM son probabilísticos
 - Ejecución del código y el uso de tokens
- El servidor del LLM puede «caerse»
- El modelo puede volverse «perezoso»

Entonces, ¿cómo es el flujo si no sé
qué modelo pedir?

Pipeline que incluye modelos predictivos

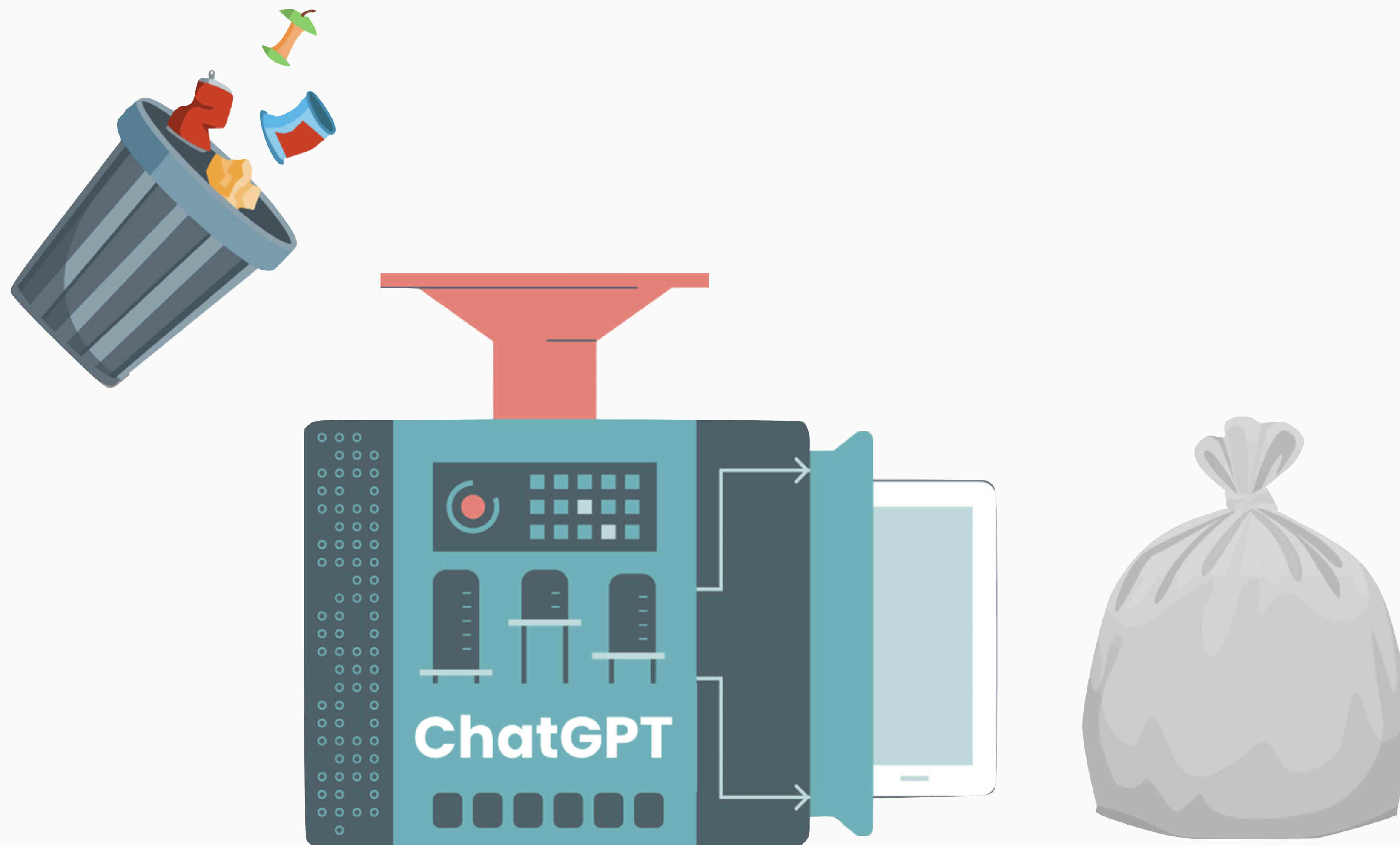
- Preámbulo → EDA (todas las variables) → Limpieza de datos → Análisis descriptivo → Analítica descriptiva → Analítica diagnóstica → Decidir qué quiere predecir y qué quiere usar para predecirlo → Pedir al LLM que sugiera modelos de ML apropiados → Construcción de todos los modelos → Comparación de los modelos → Selección del modelo ideal → Predicción usando el modelo → Ajuste de gráficos → Asistencia con la interpretación

III. Programación con LLM

No necesitamos aprender a programar en distintos lenguajes para obtener código que funciona, necesitamos convertirnos en buenos «*managers*» o «solicitantes» de código.

Piense en el LLM como un pasante que sabe programar pero necesita instrucciones muy claras. O como un traductor multilingüe que puede convertir sus prompts de lenguaje natural al lenguaje de las computadoras.

Idea clave



Condición necesaria: instrucciones claras y estructuradas

Meta/Objetivo claro

❌ «Escribe código para mis datos».

✅ «Escribe un script en Python que lea una lista de nombres de un archivo de texto, un nombre por línea, y luego imprima cada nombre en orden alfabético».

Descripción de la entrada

¿En qué formato está?

Por ejemplo, «una lista de números», «un párrafo de texto», «un archivo CSV con las columnas 'Nombre' y 'Correo electrónico'».

Salida esperada

¿Qué aspecto debería tener el resultado?

Por ejemplo, «un único número que represente el total», «un nuevo archivo de texto con nombres ordenados», «una frase que diga 'Hola, [Nombre]'»).

Lenguaje específico

Opcional, pero útil.

Si no lo sabe, puede pedir al LLM que sugiera un lenguaje adecuado para la tarea y el entorno en el que imagina utilizarlo.

Por ejemplo, «¿Cuál es una buena forma de hacer esto en Google Sheets?».

Límites y preferencias

«Explica el código en términos sencillos, paso a paso».

«Evita el uso de librerías complejas si es posible».

«Asegúrate de que el código maneja casos en los que la lista de entrada puede estar vacía».

¡Seguridad!

Condición necesaria: Iteración

- Pida siempre al LLM que explique el código que generó, línea por línea o en secciones
- El primer intento puede no ser perfecto
- Refinar el *prompt* basándose en los resultados del LLM: «El código está bien, pero ¿puedes hacer que también...?» o «Este código da error cuando.... Arréglalo.»

¡Pruébalo!

- **Escenario:** Usted tiene una lista de tareas sencillas para un proyecto personal y el tiempo estimado en horas que llevará cada tarea. Desea usar un script que calcule el total de horas estimadas para el proyecto y también le diga cuál es la tarea más larga.
- **Proceso:**
 - Instrucciones usando la estructura detallada
 - ¡No olvide pedir que explique el código!
 - Refinamiento

«¿Y cómo sabemos que el código funciona como estamos esperando?»

Pregunta típica

Verificación del código (sin saber
programar)

Los LLM son poderosos pero no infalibles. Pueden malinterpretar, cometer errores o producir código que funciona para la mayoría de los casos pero falla en casos frontera.

Principio «*Trust, but verify*»

**Objetivo para no programadores:
no es depurar el código, sino
confirmar que el código hace
funcionalmente lo que se pidió
con las entradas dadas.**

Estrategias de verificación para no programadores

- **Revisión de la explicación del LLM**
 - ¿Coincide la explicación del código del LLM con su solicitud original y comprensión de la tarea?
 - Si no coincide, puede que el código sea incorrecto o que necesite aclarar su petición

Estrategias de verificación para no programadores

- «*Black box testing*» (simplificado)
 - Concepto: No es necesario ver el interior de la caja (el código) para comprobar si funciona. Proporcione una entrada y compruebe si la salida es correcta
 - Pasos:
 - Prepare casos de prueba: Cree algunos ejemplos de entrada sencillos
 - Prediga los resultados esperados: Para sus casos de prueba simples, calcule o determine manualmente cuál debería ser la salida
 - Ejecute el código (aprovechando la ejecución del LLM): Aquí es donde herramientas de ejecución u otros entornos integrados de Python son clave
 - Compare: ¿Coincide la salida de código del LLM con la salida predicha manualmente?

Estrategias de verificación para no programadores

- **Sanity checks**

- ¿La salida tiene sentido lógico en el contexto del problema?
 - Por ejemplo, si se calcula una puntuación media, ¿está dentro del rango posible de puntuaciones? Si se calculan los costos totales, ¿es una cifra demasiado alta o baja?.

- **Prueba de casos frontera**

- Piense en entradas inusuales pero posibles
- Pregunte al LLM: «¿Cómo actuaría el código si...?» o «¿Qué sucede si...?». Luego, Pruébalo.
- Inspección visual (para resultados visuales)
- Si el código genera gráficos o representaciones visuales, ¿lucen correctos? ¿Son precisas las etiquetas?

Que NO se espera que haga

- Leer y entender cada línea de sintaxis compleja de código ✗
- Depurar mensajes de error crípticos ✗
- Convertirse en tester de software ✗

¡Pruebe con un ejemplo simple!

Epílogo. Segundo producto de
datos del portafolio

Cómo vamos a trabajar el segundo *data product*

- Con el **LLM de su elección**:
 - Brinde el contexto:
 - Caso; Sensing, seizing, configuring; Marco de OKRs, KPIs, y Acciones
 - Puede complementar con *Deep Research*
 - Realice el trabajo de datos inicial:
 - Cree smart KPI/OKR
 - **Incluya un modelo de predicción sencillo para alguna variable**
- Con **Claude**:
 - Elabore un *dashboard* interactivo a partir de:
 - Datos (con valores de predicción)
 - KPI/OKR
 - Comparta el enlace en un documento de texto

Q&A





Photo Credits

Getty Images
CX Insight (Unsplash)
Adobe Stock
Michal Vrba (Unsplash)

Mohamed Nohassi (Unsplash)
Taras Bakusevych (Medium)
Datacamp

También te puede gustar...

